

"""

photocouple-Sys-dachuang: 光伏双反应耦合智能控制系统

Open Source Project | MIT License

Copyright (c) 2026 Photocouple Ji Haixia, Liu Tianbao, Pan Junhao, Qian Huining, Shan Jiaming Team

Website: <https://github.com/060612pjh060612/photocouple-sys-dachuang>.git

Version: v1.0.1 Stable

"""

```
import random
import time
import threading
import os
import json
import csv
from datetime import datetime
from collections import deque
from typing import Dict, List, Optional, Callable, Any
from dataclasses import dataclass, asdict, fields
from enum import Enum
import logging
```

#全局系统常量与基础枚举定义

```
class SystemMode(Enum):
```

```
    SIMULATION = "simulation"
```

```
    HARDWARE = "hardware"
```

```
    CALIBRATION = "calibration"
```

```
class ReactionType(Enum):
```

```
    METHANE_REFORM = "methane_reform"
```

```
    STARCH_SYNTHESIS = "starch_synthesis"
```

```
class FaultLevel(Enum):
```

```
    INFO = 1
```

```
    WARNING = 2
```

```
    ERROR = 3
```

```
    CRITICAL = 4
```

```
SYSTEM_VERSION = "v1.0.1"
```

```
DEFAULT_CONFIG_PATH = "./config/system_config.json"
```

```
DEFAULT_LOG_PATH = "./logs/system_running.log"
```

```
DEFAULT_DATA_SAVE_PATH = "./data/runtime_history.csv"
```

#初始化全局目录

```
for dir_path in ["./config", "./logs", "./data", "./output"]:
```

```

    if not os.path.exists(dir_path):
        os.makedirs(dir_path, exist_ok=True)

#全局线程安全锁体系
GLOBAL_LOCKS = {
    "print": threading.Lock(),
    "state": threading.Lock(),
    "data": threading.Lock(),
    "config": threading.Lock()
}

#全局停机保护标记，防止多线程重复触发停机
SHUTDOWN_TRIGGERED: bool = False

#辅助函数：Enum 序列化/反序列化 修复 JSON 存储 BUG
def enum_to_dict(obj: Any) -> Dict[str, Any]:
    d = asdict(obj)
    for key, val in d.items():
        if isinstance(val, Enum):
            d[key] = val.value
    return d

def dict_to_dataclass_with_enum(data: Dict[str, Any], cls: type) -> Any:
    kwargs = {}
    for f in fields(cls):
        fname = f.name
        if fname not in data:
            continue
        raw_val = data[fname]
        if issubclass(f.type, Enum):
            kwargs[fname] = f.type(raw_val)
        else:
            kwargs[fname] = raw_val
    return cls(**kwargs)

#数据类定义：全系统结构化数据建模
@dataclass
class ReactionParam:
    light_min: float
    opt_left: float
    opt_right: float
    light_max: float
    name: str
    unit: str

```

```
description: str
```

```
@dataclass
```

```
class RuntimeState:
```

```
    raw_light: float = 0.0          # 传感器原始光强
    current_light: float = 0.0      # 经过聚散光后的有效光强
    ldr_current: float = 0.0002
    ldr_resistance: float = 50.0
    methane_eff: float = 0.0
    starch_eff: float = 0.0
    focus_multiple: float = 1.0
    disperse_ratio: float = 0.0
    system_uptime: float = 0.0
    fault_count: int = 0
    last_update_ts: float = time.time()
```

```
@dataclass
```

```
class FaultRecord:
```

```
    ts: str
    level: FaultLevel
    code: int
    message: str
    reaction: Optional[ReactionType] = None
```

```
@dataclass
```

```
class SystemConfig:
```

```
    vcc: float = 380.0
    r_fixed: float = 1000.0
    light_target_min: float = 600.0
    light_target_max: float = 1200.0
    max_focus_multiple: float = 10.0
    max_disperse_ratio: float = 0.9
    sample_interval_ms: int = 1000
    log_save_interval_s: int = 5
    max_history_length: int = 3600
    system_mode: SystemMode = SystemMode.SIMULATION
    enable_web_board: bool = False
    web_port: int = 8080
    enable_hardware_fallback: bool = True
    auto_restart_on_fault: bool = False
```

```
#配置管理模块：热加载、持久化、校验功能（修复 Enum 序列化）
```

```
class ConfigManager:
```

```
    def __init__(self, config_path: str):
```

```

self.config_path = config_path
self._config: SystemConfig = SystemConfig()
self._backup_config: SystemConfig = SystemConfig()
self._load_config()

def _load_config(self) -> None:
    with GLOBAL_LOCKS["config"]:
        if os.path.exists(self.config_path):
            try:
                with open(self.config_path, "r", encoding="utf-8") as f:
                    data = json.load(f)
                    self._config = dict_to_dataclass_with_enum(data, SystemConfig)
                    self._backup_config = dict_to_dataclass_with_enum(data,
SystemConfig)
            except Exception as e:
                logging.warning(f"Load config failed:{str(e)}, generate default
config")

                self._save_default_config()
            else:
                self._save_default_config()

def _save_default_config(self) -> None:
    self.save_config()
    logging.info("Default system config file generated")

def save_config(self) -> None:
    with GLOBAL_LOCKS["config"]:
        export_dict = enum_to_dict(self._config)
        with open(self.config_path, "w", encoding="utf-8") as f:
            json.dump(export_dict, f, indent=2, ensure_ascii=False)

def update_config(self, **kwargs) -> bool:
    with GLOBAL_LOCKS["config"]:
        try:
            for k, v in kwargs.items():
                if hasattr(self._config, k):
                    setattr(self._config, k, v)
            self.validate_config()
            self.save_config()
            return True
        except Exception as e:
            logging.error(f"Config update failed: {str(e)}")
            self._config =
dict_to_dataclass_with_enum(enum_to_dict(self._backup_config), SystemConfig)

```

```

        return False

    def validate_config(self) -> None:
        assert self._config.light_target_min < self._config.light_target_max, "Target
light range invalid"
        assert self._config.vcc > 0, "VCC voltage must be positive"
        assert 0 <= self._config.max_disperse_ratio <= 1.0, "Disperse ratio must be between
0-1"
        assert self._config.max_history_length > 0, "max_history_length >0"

    @property
    def config(self) -> SystemConfig:
        with GLOBAL_LOCKS["config"]:
            return self._config

#全局配置实例
g_config_mgr = ConfigManager(DEFAULT_CONFIG_PATH)

#日志与数据持久化模块
class SystemLogger:
    def __init__(self, log_path: str):
        self.logger = logging.getLogger("photocouple_sys")
        self.logger.setLevel(logging.INFO)
        #避免重复 handler
        self.logger.handlers.clear()
        formatter = logging.Formatter('%(asctime)s | %(levelname)s | %(message)s')

        file_handler = logging.FileHandler(log_path, encoding="utf-8")
        file_handler.setFormatter(formatter)
        stream_handler = logging.StreamHandler()
        stream_handler.setFormatter(formatter)

        self.logger.addHandler(file_handler)
        self.logger.addHandler(stream_handler)

    def log_fault(self, level: FaultLevel, code: int, message: str, reaction:
Optional[ReactionType] = None) -> None:
        record = FaultRecord(
            ts=datetime.now().isoformat(),
            level=level,
            code=code,
            message=message,
            reaction=reaction
        )

```

```

log_func = {
    FaultLevel.INFO: self.logger.info,
    FaultLevel.WARNING: self.logger.warning,
    FaultLevel.ERROR: self.logger.error,
    FaultLevel.CRITICAL: self.logger.critical
}[level]
log_func(f"FAULT-{code} | {message} | Reaction: {reaction.value if reaction else
'GLOBAL'}")
global runtime_state
with GLOBAL_LOCKS["state"]:
    runtime_state.fault_count += 1
    fault_history.append(record)

```

#全局日志实例

```

sys_logger = SystemLogger(DEFAULT_LOG_PATH)
runtime_state = RuntimeState()
fault_history: deque[FaultRecord] = deque(maxlen=1000)
light_history: deque[float] = deque()
efficiency_history: deque[Dict] = deque()

```

```

def reset_history_deques():
    """根据配置重新构建环形队列，支持 max_history_length 配置更新"""
    cfg = g_config_mgr.config
    light_history.clear()
    efficiency_history.clear()
    light_history.maxlen = cfg.max_history_length
    efficiency_history.maxlen = cfg.max_history_length

```

```
reset_history_deques()
```

```

class DataPersistence:
    def __init__(self, save_path: str):
        self.save_path = save_path
        self.last_save_ts = time.time()
        self._init_csv()

    def _init_csv(self) -> None:
        if not os.path.exists(self.save_path):
            with open(self.save_path, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow([
                    "timestamp", "raw_light", "effective_light", "ldr_current",
                    "ldr_resistance",
                    "methane_efficiency", "starch_efficiency", "focus_multiple",

```

```

        "disperse_ratio"
    ])

def save_runtime_data(self, state: RuntimeState) -> None:
    now = time.time()
    if now - self.last_save_ts < g_config_mgr.config.log_save_interval_s:
        return
    try:
        with GLOBAL_LOCKS["data"]:
            with open(self.save_path, "a", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow([
                    datetime.now().isoformat(),
                    round(state.raw_light, 2),
                    round(state.current_light, 2),
                    round(state.ldr_current, 6),
                    round(state.ldr_resistance, 2),
                    round(state.methane_eff, 2),
                    round(state.starch_eff, 2),
                    round(state.focus_multiple, 1),
                    round(state.disperse_ratio, 2)
                ])
            light_history.append(state.current_light)
            efficiency_history.append({
                "methane": state.methane_eff,
                "starch": state.starch_eff
            })
            self.last_save_ts = now
    except Exception as e:
        sys_logger.log_fault(FaultLevel.ERROR, 3001, f"CSV save error:{str(e)}")

```

```
data_persist = DataPersistence(DEFAULT_DATA_SAVE_PATH)
```

#硬件抽象层：兼容模拟/真实硬件双模式

```

class HardwareAdapter:
    def __init__(self, mode: SystemMode):
        self.mode = mode
        self._hardware_connected = False
        self._sim_noise_amp = 0.0001

    def connect(self) -> bool:
        if self.mode == SystemMode.SIMULATION:
            self._hardware_connected = True
            sys_logger.logger.info("Simulation hardware adapter connected")

```

```

        return True
    elif self.mode == SystemMode.HARDWARE:
        try:
            #此处替换真实 GPIO/USB 串口硬件初始化代码
            self._hardware_connected = True
            sys_logger.logger.info("Real LDR hardware adapter connected")
            return True
        except Exception as e:
            sys_logger.log_fault(FaultLevel.ERROR, 1001, f"Hardware connection
failed: {str(e)}")
            if g_config_mgr.config.enable_hardware_fallback:
                self.mode = SystemMode.SIMULATION
                sys_logger.log_fault(FaultLevel.WARNING, 1002, "Auto fallback to
simulation mode")
                return True
            return False
    return False

def read_ldr_current(self) -> float:
    if not self._hardware_connected:
        return 0.0002
    if self.mode == SystemMode.SIMULATION:
        return random.uniform(0.0002, 0.004)
    else:
        try:
            # 真实硬件读取占位
            raw_current = random.uniform(0.0002, 0.004)
            return max(raw_current, 0.0002)
        except Exception as e:
            sys_logger.log_fault(FaultLevel.ERROR, 1003, f"LDR read error: {str(e)}")
            return 0.0002

def current_to_ldr_resistance(self, current: float) -> float:
    if current <= 0:
        current = 0.0002
    total_R = g_config_mgr.config.vcc / current
    r_ldr = total_R - g_config_mgr.config.r_fixed
    r_ldr = max(r_ldr, 50)
    return round(r_ldr, 2)

def ldr_res_to_light(self, r_ldr: float) -> float:
    light = 1220 - (r_ldr / 160)
    return round(max(light, 50.0), 2)

```

```
hardware_adapter = HardwareAdapter(g_config_mgr.config.system_mode)
```

#光强算法模块：聚散光闭环控制（输出调节参数，不修改 state）

```
class LightRegulator:
    def __init__(self):
        self.last_adjust_ts = 0.0
        self.adjust_cooldown_s = 0.5
        self.last_adjust_mode = "idle"

    def adjust_light(self, current_raw_light: float) -> Dict:
        cfg = g_config_mgr.config
        if current_raw_light <= 0.0:
            self.last_adjust_mode = "off"
            return {
                "status": "无光照 | 聚散系统关闭",
                "type": "shutdown",
                "focus_multiple": 1.0,
                "disperse_ratio": 0.0
            }

        if current_raw_light < cfg.light_target_min:
            diff = cfg.light_target_min - current_raw_light
            focus_multiple = round(1 + diff / 150, 1)
            focus_multiple = min(focus_multiple, cfg.max_focus_multiple)
            disperse_ratio = 0.0
            self.last_adjust_mode = "focus"
            return {
                "status": f"聚光调控 | 倍率: {focus_multiple}倍",
                "type": "focus",
                "focus_multiple": focus_multiple,
                "disperse_ratio": disperse_ratio
            }

        elif current_raw_light > cfg.light_target_max:
            diff = current_raw_light - cfg.light_target_max
            disperse_ratio = round(diff / 200, 2)
            disperse_ratio = min(disperse_ratio, cfg.max_disperse_ratio)
            focus_multiple = 1.0
            self.last_adjust_mode = "disperse"
            return {
                "status": f"散光调控 | 衰减率: {disperse_ratio * 100:.1f}%",
                "type": "disperse",
                "focus_multiple": focus_multiple,
                "disperse_ratio": disperse_ratio
            }
}
```

```

else:
    focus_multiple = 1.0
    disperse_ratio = 0.0
    self.last_adjust_mode = "idle"
    return {
        "status": "光强理想 | 聚散系统休眠, 稳态运行",
        "type": "idle",
        "focus_multiple": focus_multiple,
        "disperse_ratio": disperse_ratio
    }

```

```
light_regulator = LightRegulator()
```

#反应单元核心逻辑

```

reaction_params: Dict[ReactionType, ReactionParam] = {
    ReactionType.METHANE_REFORM: ReactionParam(
        light_min=200, opt_left=600, opt_right=1000, light_max=1200,
        name="甲烷光催化重整制氢", unit="m³/h", description="利用光照驱动甲烷分解生成氢气与高附加值碳产物"
    ),
    ReactionType.STARCH_SYNTHESIS: ReactionParam(
        light_min=200, opt_left=500, opt_right=1200, light_max=1500,
        name="人工酶促淀粉合成", unit="g/h", description="电驱动辅酶循环耦合暗酶反应合成直链淀粉"
    )
}

```

```

def calc_reaction_efficiency(light: float, r_type: ReactionType) -> float:
    param = reaction_params[r_type]
    if light <= 0.0:
        return 0.0
    if light < param.light_min:
        if r_type == ReactionType.METHANE_REFORM:
            return round(random.uniform(8.0, 15.0), 2)
        else:
            return round(random.uniform(5.0, 12.0), 2)
    elif light < param.opt_left:
        if r_type == ReactionType.METHANE_REFORM:
            return round(random.uniform(18.0, 28.0), 2)
        else:
            return round(random.uniform(14.0, 22.0), 2)
    elif param.opt_left <= light <= param.opt_right:
        if r_type == ReactionType.METHANE_REFORM:
            return round(random.uniform(35.0, 42.0), 2)

```

```

        else:
            return round(random.uniform(26.0, 34.0), 2)
    elif light <= param.light_max:
        if r_type == ReactionType.METHANE_REFORM:
            return round(random.uniform(25.0, 33.0), 2)
        else:
            return round(random.uniform(12.0, 20.0), 2)
    else:
        return round(random.uniform(10.0, 20.0), 2)

def get_reaction_status(light: float, r_type: ReactionType) -> str:
    param = reaction_params[r_type]
    if r_type == ReactionType.METHANE_REFORM:
        if light <= 0.0:
            return "完全无光 | 反应暂停"
        elif light < param.light_min:
            return "严重弱光 | 低载待机"
        elif light < param.opt_left:
            return "偏弱光 | 聚光补强反应"
        elif param.opt_left <= light <= param.opt_right:
            return "理想光强 | 高效重整制氢"
        elif light <= param.light_max:
            return "偏强光 | 降负荷稳压"
        else:
            return "光强超限 | 过热保护"
    else:
        if light <= 0.0:
            return "完全无光 | 酶促反应暂停"
        elif light < param.light_min:
            return "极弱光 | 全开辅助聚光"
        elif light < param.opt_left:
            return "弱光 | 低倍率聚光补能"
        elif param.opt_left <= light <= param.opt_right:
            return "标准理想光强 | 酶促避光合成"
        else:
            return "强光过载 | 多级散光衰减"

#全局运行开关
system_running = True

def trigger_global_shutdown(reaction: Optional[ReactionType], msg: str):
    """安全触发全局停机，只执行一次"""
    global SHUTDOWN_TRIGGERED, system_running
    with GLOBAL_LOCKS["state"]:

```

```

        if SHUTDOWN_TRIGGERED:
            return
        SHUTDOWN_TRIGGERED = True
        system_running = False
    with GLOBAL_LOCKS["print"]:
        print(f"\n[CRITICAL ALERT] {msg}")
    sys_logger.log_fault(FaultLevel.CRITICAL, 2000, msg, reaction)

def reaction1_methane():
    global runtime_state
    while system_running:
        with GLOBAL_LOCKS["state"]:
            light = runtime_state.current_light
            fm = runtime_state.focus_multiple
            dr = runtime_state.disperse_ratio
            status_msg = get_reaction_status(light, ReactionType.METHANE_REFORM)

            if light <= 0.0:
                trigger_global_shutdown(ReactionType.METHANE_REFORM, "甲烷反应无有效光照，
触发全局停机")
                break

            with GLOBAL_LOCKS["print"]:
                print(f"[甲烷重整]有效光强: {light:.2f} W/m² | 利用率:
{runtime_state.methane_eff:.2f}% | 倍率: {fm:.1f}x 衰减: {dr:.2f} | {status_msg}")
                time.sleep(1)

def reaction2_starch():
    global runtime_state
    while system_running:
        with GLOBAL_LOCKS["state"]:
            light = runtime_state.current_light
            fm = runtime_state.focus_multiple
            dr = runtime_state.disperse_ratio
            mode_msg = get_reaction_status(light, ReactionType.STARCH_SYNTHESIS)

            if light <= 0.0:
                trigger_global_shutdown(ReactionType.STARCH_SYNTHESIS, "淀粉合成无有效光照，
触发全局停机")
                break

            with GLOBAL_LOCKS["print"]:
                print(f"[淀粉合成]有效光强: {light:.2f} W/m² | 利用率:
{runtime_state.starch_eff:.2f}% | 倍率: {fm:.1f}x 衰减: {dr:.2f} | {mode_msg}\n")

```

```

        time.sleep(1)

def light_control_task():
    global runtime_state, system_running
    with GLOBAL_LOCKS["print"]:
        print("=" * 85)
        print(f"    光伏双反应耦合控制系统 {SYSTEM_VERSION}    ")
        print("=" * 85)
        print(f"双反应理想光强区间:
{g_config_mgr.config.light_target_min}~{g_config_mgr.config.light_target_max} W/m²")
        print(f"系统运行模式: {g_config_mgr.config.system_mode.value.upper()}")
        print("=" * 85 + "\n")
        sys_logger.logger.info("System startup complete, all threads starting")

start_ts = time.time()
try:
    while system_running:
        try:
            I = hardware_adapter.read_ldr_current()
            R_ldr = hardware_adapter.current_to_ldr_resistance(I)
            raw_light = hardware_adapter.ldr_res_to_light(R_ldr)

            # 只在这里执行一次光强调节
            adj_result = light_regulator.adjust_light(raw_light)
            f_mult = adj_result["focus_multiple"]
            d_ratio = adj_result["disperse_ratio"]
            # 计算经过聚散光后的有效光强
            effective_light = raw_light * f_mult * (1.0 - d_ratio)

            methane_eff = calc_reaction_efficiency(effective_light,
ReactionType.METHANE_REFORM)
            starch_eff = calc_reaction_efficiency(effective_light,
ReactionType.STARCH_SYNTHESIS)

            with GLOBAL_LOCKS["state"]:
                runtime_state.ldr_current = I
                runtime_state.ldr_resistance = R_ldr
                runtime_state.raw_light = raw_light
                runtime_state.current_light = effective_light
                runtime_state.focus_multiple = f_mult
                runtime_state.disperse_ratio = d_ratio
                runtime_state.methane_eff = methane_eff
                runtime_state.starch_eff = starch_eff
                runtime_state.system_uptime = time.time() - start_ts

```

```

        runtime_state.last_update_ts = time.time()

        with GLOBAL_LOCKS["print"]:
            print(f"[底层监测] 电流: {I:.6f} A | LDR 电阻: {R_ldr} Ω | 原始光强: {raw_light:.2f} W/m² | 有效光强: {effective_light:.2f} W/m²")

        data_persist.save_runtime_data(runtime_state)

    except Exception as e:
        sys_logger.log_fault(FaultLevel.ERROR, 4001, f"Sampling loop exception: {str(e)}")

        if g_config_mgr.config.auto_restart_on_fault:
            pass  # 可扩展采样循环内部重启逻辑

    time.sleep(g_config_mgr.config.sample_interval_ms / 1000)

    with GLOBAL_LOCKS["print"]:
        print("\n[系统终止] 安全联锁触发, 全部反应单元强制停止")
        sys_logger.logger.info("System full shutdown triggered by safety interlock")
except KeyboardInterrupt:
    with GLOBAL_LOCKS["print"]:
        print("\n[手动终止] 收到 Ctrl+C, 系统停止")
    sys_logger.logger.info("System manual stop via Ctrl+C")
    system_running = False

# 系统健康诊断与辅助工具模块
class SystemDiagnosis:
    def get_health_score(self) -> float:
        if len(light_history) < 10:
            return 100.0
        window = list(light_history)[-60:]
        ideal_min = g_config_mgr.config.light_target_min
        ideal_max = g_config_mgr.config.light_target_max
        ideal_count = sum(1 for x in window if ideal_min <= x <= ideal_max)
        health_ratio = ideal_count / len(window) * 100
        return round(health_ratio, 1)

    def export_diagnosis_report(self) -> Dict:
        avg_light = round(sum(light_history)/len(light_history), 2) if light_history else 0
        avg_meth = round(sum(x["methane"] for x in efficiency_history)/len(efficiency_history), 2) if efficiency_history else 0
        avg_starch = round(sum(x["starch"] for x in efficiency_history)/len(efficiency_history), 2) if efficiency_history else 0

```

```

    return {
        "version": SYSTEM_VERSION,
        "uptime": round(runtime_state.system_uptime, 2),
        "health_score": self.get_health_score(),
        "total_faults": runtime_state.fault_count,
        "recent_1h_light_avg": avg_light,
        "recent_methane_eff_avg": avg_meth,
        "recent_starch_eff_avg": avg_starch
    }

diagnosis = SystemDiagnosis()

#系统主入口
if __name__ == "__main__":
    if not hardware_adapter.connect():
        sys_logger.log_fault(FaultLevel.CRITICAL, 9999, "Hardware init failed, system
exit")
        exit(1)

    t_light = threading.Thread(target=light_control_task, daemon=True)
    t_reform = threading.Thread(target=reaction1_methane, daemon=True)
    t_starch = threading.Thread(target=reaction2_starch, daemon=True)

    t_light.start()
    t_reform.start()
    t_starch.start()

    try:
        while system_running:
            time.sleep(0.2)
    finally:
        system_running = False
        t_light.join(timeout=2)
        t_reform.join(timeout=2)
        t_starch.join(timeout=2)
        with GLOBAL_LOCKS["print"]:
            print("\n===== 双反应耦合系统 全部终止完成 =====")
            report = diagnosis.export_diagnosis_report()
            print(f"系统运行诊断报告: {json.dumps(report, indent=2,
ensure_ascii=False)}")
            sys_logger.logger.info(f"System exit, final report exported")

```